

Chapter 5: Pointers and Arrays

Relational graph edition. Pull-shaped. Top-down.

How to use this document

This is the hardest chapter in K&R. Every section builds on the last. Do not skip problems. Red = checkpoint. Green = practice. Blue = target.

Big picture

A pointer is a variable that holds the address of another variable. Dereferencing a pointer follows that address to reach the value. Array names decay to pointers. Pointer arithmetic moves through memory in strides of `sizeof(type)`. This chapter is the core of C -- everything else (strings, structs, dynamic memory, `argv`) builds on it.

Target problem

Write a program that sorts command-line arguments (`argv`) alphabetically using a sort function that takes a pointer-to-function comparator, demonstrating: `argv` as pointer array, pointer arithmetic, and function pointers.

The Graph

Node inventory

Node	Syntax	What it holds
pointer variable	<code>int *p;</code>	An address (memory location of another variable)
address-of	<code>&x</code>	Produces the address where <code>x</code> is stored
dereference	<code>*p</code>	Follows the address in <code>p</code> to get the value there
array name	<code>int a[10];</code> then <code>a</code>	Decays to pointer to first element (<code>&a[0]</code>)
NULL	<code>(void *)0</code>	Invalid address -- means 'points to nothing'
void *	<code>void *p;</code>	Generic pointer -- can hold any address, must cast to use
pointer arithmetic	<code>p + n</code>	Moves <code>n * sizeof(*p)</code> bytes forward
<code>argv</code> / <code>argc</code>	<code>char *argv[]</code>	Array of string pointers from command line
function pointer	<code>int (*fp)(int, int)</code>	Holds address of a function; call via <code>(*fp)(args)</code>

sizeof	sizeof(type)	Returns size in bytes -- the stride for pointer arithmetic
--------	--------------	--

Edge inventory

Edge	Rule
E1: & produces pointer	&x gives you a pointer to x. Type of &x is 'pointer to type-of-x'
E2: * follows pointer	*p gives you the value at the address stored in p
E3: array decays	Array name used in expression becomes pointer to element 0. $a[i] == *(a+i)$
E4: pointer + int	$p+n$ moves forward n elements ($n * \text{sizeof}(*p)$ bytes). $p-n$ moves back
E5: pointer - pointer	$p-q$ gives the number of elements between two pointers (same array only)
E6: function pointer	Function name without () is a pointer to that function. Call via $fp(args)$ or $(*fp)(args)$
E7: argv structure	$argv[0]$ = program name, $argv[1..argc-1]$ = arguments, $argv[argc] = \text{NULL}$

Zoom in -- one edge at a time

Edge 1: Pointers and addresses

Every variable lives at a specific memory address. `&x` gives you that address. A pointer variable stores an address. The declaration `int *p;` means 'p is a variable that holds the address of an int.'

Memory layout example:

Variable	Address	Value
int x	0x1000	42
int *p	0x1008	0x1000 (address of x)

```
int x = 42;
int *p = &x; /* p holds the address of x */
printf("%d\n", *p); /* follows p to get 42 */
*p = 99; /* changes x through p */
printf("%d\n", x); /* prints 99 */
```

Problem 5.1: If `int a=5, b=10, *p=&a, *q=&b;`, what does `*p + *q` evaluate to?

Answer: 15. `*p` is 5 (value at a), `*q` is 10 (value at b). Sum is 15.

Edge 2: Pointers and function arguments (the swap problem)

C is call-by-value. To let a function modify the caller's variable, pass a pointer to it. The function receives a copy of the pointer (the address), and uses `*` to modify the original.

```
void swap(int *px, int *py) {
    int temp = *px;
    *px = *py;
    *py = temp;
}
/* Call: swap(&a, &b); -- pass addresses */
```

Problem 5.2: Write a function that takes two int pointers and sets both values to their sum.

`void add_to_both(int *a, int *b)`

Answer: `int sum = *a + *b; *a = sum; *b = sum;`

Edge 3: Arrays decay to pointers

When an array name appears in an expression, it becomes a pointer to element 0. `a[i]` is identical to `*(a + i)`. This is not a metaphor -- the compiler generates the same code for both. Exception: `sizeof(a)` gives the total array size, not pointer size.

Expression	Equivalent	Type
a	<code>&a[0]</code>	<code>int *</code> (pointer to first element)

a[3]	*(a + 3)	int (value of 4th element)
&a[3]	a + 3	int * (pointer to 4th element)

Problem 5.3: If `int a[] = {10, 20, 30, 40};` and `int *p = a;`, what is `*(p+2)`?

Answer: 30. p points to a[0], p+2 points to a[2], *(p+2) is the value 30.

CHECKPOINT A

Without looking: what is the difference between `&x` and `*p`? Why must `swap()` take pointers instead of values? What does `a[i]` actually mean in terms of pointer arithmetic?

Edge 4: Pointer arithmetic

$p + n$ moves forward n elements, not n bytes. If p is `int *` and `sizeof(int)` is 4, then $p+1$ advances 4 bytes. The compiler multiplies by `sizeof(*p)` automatically. Subtracting two pointers gives element count between them.

If $p = 0x1000$ (<code>int *</code>)	Expression	Address
	p	0x1000
	$p + 1$	0x1004 (moved 4 bytes)
	$p + 3$	0x100C (moved 12 bytes)

Problem 5.4: If `double *dp` is at address 0x2000, what is the address of $dp+5$? (`sizeof(double) = 8`)

Answer: $0x2000 + 5 \cdot 8 = 0x2028$.

Edge 5: Character pointers and strings

A string literal "hello" is stored as a char array with `\0` terminator. A `char *` can point to the first character. String functions like `strcpy/strcmp` walk the pointer forward until they hit `\0`.

```
/* strcpy using pointers */
void my_strcpy(char *dest, char *src) {
    while ((*dest++ = *src++) != '\0')
        ;
}
```

Parsing $*dest++ = *src++$: (1) copy `*src` to `*dest`, (2) increment both pointers, (3) test if copied char was `\0`. Three operations in one expression.

Problem 5.5: Write `int my_strcmp(char *s, char *t)` that returns <0 , 0 , or >0 .

Answer: `for (; *s == *t; s++, t++) if (*s == '\0') return 0; return *s - *t;`

Edge 6: Pointer arrays and argv

`char *argv[ ]` is an array of pointers-to-char. Each element points to a null-terminated string. `argv[0]` is the program name. `argv[argc]` is `NULL`. This is how the OS passes command-line arguments to your program.

Index	argv[i]	Points to
0	argv[0]	"./myprogram" (program name)
1	argv[1]	"hello" (first argument)
2	argv[2]	"world" (second argument)

3	argv[3]	NULL (sentinel)
Problem 5.6: Write a program that prints command-line arguments one per line, using a pointer instead of an index. Answer: <code>while (*++argv) printf("%s\n", *argv);</code> (skip argv[0], print until NULL)		
CHECKPOINT B Write <code>int my_strlen(char *s)</code> using pointer arithmetic only (no indexing, no counter variable). Hint: remember where you started.		

Edge 7: Function pointers

A function name without () is a pointer to that function. You can store it in a variable and call it later. This enables polymorphism: pass different comparison functions to the same sort routine.

```
/* Function pointer declaration */
int (*compare)(const char *, const char *);

/* Assign and call */
compare = strcmp;
int result = compare("abc", "def"); /* calls strcmp */

/* Reading the declaration: */
/* (*compare) = "compare is a pointer" */
/* (*compare)(...) = "to a function taking ..." */
/* int (*compare)(...) = "returning int" */
```

Problem 5.7: Declare a function pointer fp that points to a function taking two ints and returning void.

Answer: void (*fp)(int, int);

Solving the target problem

Walk: E6 (argv as pointer array) + E5 (strings) + E7 (function pointer for comparison) + E4 (pointer arithmetic in sort)

```
#include <stdio.h>
#include <string.h>

void sort_strings(char *arr[], int n,
int (*cmp)(const char *, const char *)) {
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (cmp(arr[i], arr[j]) > 0) {
                char *tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
            }
}

int main(int argc, char *argv[]) {
    sort_strings(argv + 1, argc - 1, strcmp);
    for (int i = 1; i < argc; i++)
        printf("%s\n", argv[i]);
    return 0;
}
```

Edges walked: E6 (argv+1 skips program name), E7 (strcmp passed as function pointer), E5 (string comparison), E3 (arr[i] is pointer dereference).

CHECKPOINT C

Modify the program to accept a -r flag for reverse sort. When -r is present, pass a reverse_strcmp function instead of strcmp.

Final exam

Problem 1: Write `void strcat(char *s, char *t)` using pointer arithmetic (no indexing).

Problem 2: Write a function that returns a pointer to the last occurrence of char `c` in string `s`, or `NULL` if not found.

Problem 3: Explain the difference between `char s[]` and `char *s` when initialized with a string literal.

Problem 4: Write `int day_of_year(int year, int month, int day)` using a 2D array of month lengths.

Problem 5: Decode this declaration: `char (*(*x())[ ] )();`

CHECKPOINT D

Solve at least 3. Report in chat.

Answer key

1. Walk E4+E5: advance s to end (`while(*s) s++`), then copy t (`while((*s++ = *t++))`).
2. Walk E4+E5: scan from start, remember last position where `*s==c`. Return that pointer or NULL.
3. `char s[] = "hello"` creates a modifiable 6-byte array. `char *s = "hello"` creates a pointer to a read-only string literal. Modifying via `s[0]='H'` is legal for the array, undefined behavior for the pointer.
4. Walk E3: static `int daytab[2][13]` with leap/non-leap month lengths. Sum months 1 through month-1, add day.
5. x is a function returning a pointer to an array of pointers to functions returning char. (Use the spiral/clockwise rule: start at x, go right for `()`, left for `*`, right for `[]`, etc.)